

Parquivo

Cómo está construida la aplicación

Informe técnico · 25 de agosto de 2026

Sistema de gestión para parqueaderos · CQS Labs

1. Resumen

Parquivo es una aplicación web para administrar parqueaderos. Un mismo sistema atiende a muchos establecimientos a la vez, cada uno con sus tarifas, sus horarios, sus turnos, sus convenios y su propio historial, sin que ninguno pueda ver los datos de otro.

El producto está construido alrededor de una idea: **la taquilla es la ruta crítica**. Es la pantalla donde alguien pasa el día entero, de pie, con una fila delante. Todo lo demás —configurar tarifas, declarar convenios, mirar reportes— se visita una vez al mes. Esa jerarquía decide el diseño, el rendimiento y hasta dónde se pone cada botón.

El tamaño de lo construido

Medida	Cantidad
Pantallas (rutas)	26
Componentes de interfaz	60 archivos
Módulos de lógica de negocio	76 archivos en 19 áreas
Líneas de código de aplicación	26.902
Líneas de pruebas	13.261
Pruebas automáticas	712, en 83 archivos
Migraciones de base de datos	20
Confirmaciones de cambios (commits)	166

La proporción entre código y pruebas —una línea de prueba por cada dos de aplicación— no es casual: el dinero que cobra un parqueadero pasa por este código.

2. Con qué está hecha

Pieza	Elección	Por qué
-------	----------	---------

Lenguaje	TypeScript	Los errores de tipo salen al compilar, no delante de un cliente.
Interfaz	React 19	Estándar de la industria; permite contratar quien la mantenga.
Servidor	Next.js 16 (App Router)	La página se arma en el servidor: llega dibujada y sin parpadeo.
Base de datos	PostgreSQL	Es la que sabe aislar inquilinos de verdad, con RLS.
Acceso a datos	Drizzle ORM	Consultas con tipos verificados; sin SQL suelto en cadenas.
Sesiones	better-auth	Autenticación probada, sin inventar criptografía propia.
Pruebas	Vitest	Rápido, y corre las pruebas contra una base real.

Seis dependencias en total. Es deliberado: cada librería añadida es código ajeno que hay que mantener, actualizar y auditar. No hay librería de componentes, ni de estilos, ni de gráficos — todo eso está escrito a medida y son unas dos mil líneas, menos de lo que pesaría cualquiera de esas dependencias.

3. Las cinco reglas que gobiernan el proyecto

El proyecto tiene una constitución escrita, ratificada y versionada (versión 1.1.0). No es un documento decorativo: cuando una decisión de código choca con ella, se acata la constitución o se la enmienda formalmente. Nunca se argumenta alrededor.

I. Aislamiento entre parqueaderos (no negociable)

Ningún establecimiento puede ver, ni por accidente ni por error de programación, los datos de otro. Es la única regla marcada como no negociable, y la sección 4 explica cómo se hace cumplir.

II. Configurable por establecimiento, cero valores quemados

Ninguna tarifa, horario, capacidad ni redondeo está escrito en el código. Todo se declara por establecimiento. Un parqueadero de pueblo y uno de centro comercial usan el mismo programa con configuraciones distintas.

III. La taquilla es la ruta crítica

Cada añadido a esa pantalla hay que justificarlo. Es la razón de que la taquilla no tenga menús de configuración, ni avisos de marketing, ni nada que no sirva para atender el vehículo que está delante.

IV. Integridad e inmutabilidad del historial

Un movimiento cobrado no se edita ni se borra nunca. Si hubo un error, se emite una corrección que queda al lado del original, con quién la hizo y por qué. La base de datos lo impone con disparadores, no con buena voluntad del programador.

V. Alcance deliberado

Lo que no se necesita hoy, no se construye. Es la razón de que no haya pasarela de pagos, ni aplicación móvil nativa, ni reconocimiento de placas por cámara.

4. El aislamiento entre parqueaderos

Ésta es la parte más importante del sistema y la más difícil de añadir después, así que se construyó primero, antes que ninguna pantalla.

El problema

En un sistema donde muchos clientes comparten la misma base de datos, el riesgo no es que alguien ataque: es que un programador escriba una consulta y se le olvide el «...donde el parqueadero sea el mío». Ese olvido no falla, no da error, y devuelve los datos de todo el mundo.

La solución: tres roles y dos puertas

La protección no vive en el código de la aplicación sino en **la base de datos**, con «seguridad a nivel de fila» (RLS) forzada. Trece tablas llevan políticas que filtran por establecimiento antes de devolver nada.

Rol de PostgreSQL	Para qué	Puede saltarse el filtro
app_tenant	Todo lo que hace un establecimiento	No
app_platform	El administrador general de la plataforma	Sí, y sólo él
app_migrator	Cambiar la estructura de las tablas	Sólo en migraciones

Y para llegar a los datos hay exactamente **dos puertas**, ninguna otra:

- `conAmbito(parqueaderoId, ...)` — abre una transacción con el establecimiento fijado. Si una consulta olvida el filtro, la base de datos lo aplica igual.
- `comoPlataforma(privilegio, ...)` — la puerta del administrador general. Cada uso queda registrado con el motivo, así que el privilegio deja rastro.

La consecuencia práctica: **el olvido deja de ser posible**. Aunque alguien escriba mañana una consulta descuidada, la base devuelve sólo lo del establecimiento en curso.

Cómo se comprueba

Hay **17 archivos de pruebas dedicados sólo a esto**. El patrón es siempre el mismo: se crean dos establecimientos, A y B, y se comprueba que A no alcanza absolutamente nada de B — ni sus tarifas, ni sus movimientos, ni sus operarios, ni sus convenios, ni siquiera saber si existen.

Eso último importa más de lo que parece: pedir un dato de otro establecimiento responde exactamente igual que pedir uno inexistente. Si respondiera distinto, se podría averiguar qué existe probando identificadores.

5. El modelo de datos

Diecisiete tablas. Las principales:

Tabla	Qué guarda
parqueadero	El establecimiento: nombre, código legible, ciudad, estado de la cuenta.
usuario / asignacion	Las personas y a qué parqueadero pertenecen, con qué rol.
tipo_vehiculo	Automóvil, motocicleta, bicicleta. Catálogo compartido.
tarifa	Versionada por fechas: cambiar el precio no altera lo ya cobrado.
horario / capacidad	Cuándo abre y cuántos cupos tiene, por tipo de vehículo.
turno / sesion_turno	El turno declarado, y cada vez que alguien lo abre y lo cierra.
convenio	Los descuentos acordados, con sus placas o su sello.
movimiento	Cada entrada y cada salida. El corazón del historial.
correccion	Los arreglos a un cobro cerrado, al lado del original.
auditoria	Quién hizo qué, cuándo, y desde qué privilegio.

Dos decisiones que conviene entender

La tarifa se copia dentro del movimiento al cobrar. No se guarda una referencia a la tarifa: se guarda una copia de lo que decía en ese momento. Así, subir el precio en marzo no cambia lo que se cobró en febrero, y un cobro de hace un año se puede explicar aunque la tarifa ya no exista.

Lo mismo con los convenios. Cada movimiento guarda copia del convenio aplicado. Vencer un convenio no altera ni un peso de lo ya cobrado con él.

6. Quién puede hacer qué

Hay tres roles, y los permisos se declaran en un solo archivo como datos, no dispersos en condicionales por las pantallas. Añadir una operación obliga a declarar quién puede hacerla: olvidarse produce un error al compilar, no un agujero silencioso.

Rol	Alcance
Administrador general	La plataforma entera: crea parqueaderos, cuentas, suspende servicios.
Administrador de parqueadero	Su establecimiento completo. Además puede atender en taquilla.
Operario	La taquilla y lo suyo. Ver sección siguiente.

Los límites del operario

Un operario ve:

- **Taquilla y Adentro ahora** — su trabajo.
- **Tarifas**, sólo lectura — para responder «¿cuánto me cuesta?» sin llamar a nadie.
- **Convenios**, sólo cuáles están vigentes y qué hace cada uno.
- **Últimos cobros y Reportes** — únicamente lo que él cobró, en sus turnos.
- **Mis turnos y Ajustes** — lo suyo.

No ve Mi parqueadero, ni Horario y capacidad, ni Turnos, ni Operarios. Esa última importa especialmente: la lista del equipo trae el correo de cada compañero, quién está bloqueado y quién sigue con contraseña temporal.

El recorte del menú es comodidad, no seguridad. La constitución es explícita: esconder una opción en la interfaz nunca constituye control de acceso. Cada pantalla cerrada comprueba el permiso en el servidor antes de consultar nada, así que escribir la dirección a mano tampoco funciona.

La única excepción: los permisos delegables

Corregir un cobro ya cerrado es de administradores. Pero un parqueadero pequeño tiene un operario de confianza en el turno de noche y ningún administrador des-

pierto, y negarle una corrección hasta la mañana es una regla que la operación real termina saltándose por fuera del sistema.

Por eso un administrador puede autorizar a una persona concreta, y queda registrado que él la autorizó. Es la única operación delegable que existe, y la lista está cerrada por constitución: una delegación que no figure ahí no concede nada, por mucho que alguien inserte la fila en la base.

7. La taquilla

La pantalla que se usa todo el día. Su diseño responde a una escena concreta: alguien de pie, mirando de reojo entre el carro y la pantalla, a veces con fila.

Un solo campo para todo

No hay botón de «entrada» ni de «salida». Se escribe la placa y se pulsa Enter; el sistema mira si ese vehículo ya está adentro y decide. Si no está, ofrece registrar la entrada. Si está, calcula lo que debe y ofrece cobrarlo.

Esto elimina toda una clase de error: no se puede registrar una entrada de un carro que ya entró, ni cobrar la salida de uno que nunca llegó.

Las bicicletas y las fichas

Una bicicleta no tiene placa, así que se le entrega una **ficha**, que es simplemente otro tique donde en lugar de la placa dice «Ficha 3». Al recibirla se anotan nombre, celular y cédula. Para sacarla, el operario escribe el número de ficha en el mismo campo, igual que una placa.

Cuántas fichas hay es la capacidad declarada de bicicletas: no hay inventario aparte.

El cálculo del cobro

Al resolver una salida, la pantalla muestra el total y **su desglose**, siempre. Un total que quien atiende no puede explicarle al cliente es un problema en plena fila.

- Tarifa base, por intervalos o por tarifa plena, según lo declarado.
- Los convenios que aplicaron, y los que no aplicaron con el motivo.
- Los sellos de comercio que el cliente traiga, que se marcan y recalculan al momento.
- El redondeo, si el establecimiento lo configuró.

Un parqueadero puede además decidir que los convenios **no** apliquen a bicicletas, que es lo normal, o que sí.

8. Turnos, caja e historial

Los turnos se declaran en configuración y se abren y cierran en la taquilla. Cada movimiento guarda **quién hizo cada extremo**: quién recibió el vehículo y quién lo cobró, que pueden ser personas y turnos distintos.

Ésa es la fuente de casi todos los desacuerdos al cuadrar una caja, y por eso la pantalla «Mis turnos» separa explícitamente los vehículos que quedan adentro, los que entraron en el turno propio, y las salidas que se cobraron de vehículos recibidos por otro.

El historial no se toca

Un movimiento cerrado es inmutable. La base de datos lo impone con disparadores: ni siquiera la aplicación puede modificarlo. Si hubo un error de cobro, se emite una corrección que se guarda al lado, con el importe corregido, el motivo y quién la autorizó. Los reportes usan el importe corregido; el original sigue ahí.

El código del establecimiento también es inmutable de por vida, con su propio disparador, porque aparece impreso en los tiques que ya se entregaron.

9. La interfaz

Identidad

La paleta sale del logo de CQS Labs: negro casi puro, blanco, y un azul aciano. Tres colores y nada más. Arranca en tema oscuro —una caseta se atiende de noche y una pantalla blanca a las dos de la mañana deslumbra— y cada persona puede cambiarlo.

La tipografía es **Archivo**, que viene de la señalética: se lee de pie, de reojo y con luz mala. Las cifras, placas y horas van en monoespaciada, porque en columna es lo que permite compararlas sin que el ojo pierda el renglón.

Contraste verificado

Todos los colores de texto se comprobaron con cálculo de luminancia contra su fondo, en los dos temas. Todos superan el mínimo AA de accesibilidad, y los bordes de los controles superan el 3:1 que exige la norma cuando el contorno es lo único que distingue un botón del texto.

El principio de composición

Las pantallas no encierran el contenido en tarjetas. Lo que separa es el espacio y una línea de un píxel donde de verdad hace falta un corte. La caja se reserva para lo que **es** un objeto: un control que hay que encontrar sin buscar, un aviso que interrumpe, o algo que representa una cosa del mundo real.

El ejemplo de eso último es el campo de placa de la taquilla, que va enmarcado y grande a propósito: no es un campo de formulario que casualmente es grande, es la representación de la placa del carro que el operario tiene delante.

Movimiento

Cinco duraciones y dos curvas, no más. Todas las animaciones existen para dar información —que algo está en camino, de dónde vino una cosa nueva, que una acción terminó— y ninguna es decorativa. Quien pidió menos movimiento en su sistema operativo lo pidió en serio: no se atenúan, se apagan.

10. Cómo se comprueba que funciona

Clase de prueba	Archivos	Qué comprueba
Aislamiento	17	Que un establecimiento no alcance nada de otro.
Integración	40	Reglas de negocio contra una base de datos real.
Unitarias	25	Cálculos puros y forma de las pantallas.
Herramienta	1	Que el propio andamiaje de pruebas no mienta.

Las pruebas de integración corren contra PostgreSQL de verdad, no contra una imitación. Es más lento —la suite completa tarda unos tres minutos— pero es la única manera de comprobar que las políticas de seguridad de la base hacen lo que se cree.

Además hay pruebas de guardia, que existen por errores ya cometidos: una prueba que ningún estilo escrito dentro de un elemento tape una regla de la hoja de estilos, que es un fallo que rompió cuatro pantallas antes de que existiera esa prueba.

Estado actual

712 pruebas, todas pasando. Sin errores de tipos ni de análisis estático.

11. Qué está hecho y qué falta

Módulo	Para quién	Estado
F1 · Plataforma y parqueaderos	Administrador general	Entregado
F2 · Configuración del establecimiento	Administrador	Entregado
F3 · Taquilla — vehículos	Operario	Falta el tique impreso
F3.5 · Convenios y descuentos	Administrador	Entregado
F4 · Bicicletas y fichas	Operario	Entregado
F5 · Historial y reportes	Administrador	Entregado

Lo que queda pendiente

- **La impresión térmica del tique.** El código está escrito; lo que falta es probarlo con la impresora. En la última prueba salió papel con basura, y la causa no estaba en el programa: el sistema tenía instalado el controlador de un plóter en lugar del de una impresora de tiques.
- **Las pantallas de la plataforma** —panel, parqueaderos, cuentas— todavía usan el lenguaje visual anterior al rediseño.
- **El teléfono.** Todas las pantallas funcionan en móvil, pero las 21 maquetas móviles del diseño no se han aplicado una por una.
- **Pagos y suscripciones.** Deliberadamente fuera del alcance actual. Hoy el administrador general suspende o reactiva un establecimiento a mano.

12. Cómo se desarrolla

El proyecto usa desarrollo guiado por especificación. Antes de escribir código, cada módulo pasa por: especificación, resolución de ambigüedades, plan técnico, lista de verificación, tareas, análisis de coherencia e implementación.

Suena burocrático y evita un problema concreto: que las decisiones importantes se tomen sin querer, escribiendo. Las cinco especificaciones están en el repositorio y se pueden leer.

Cada cambio queda en una confirmación con su explicación de **por qué**, no sólo de qué se cambió. Esas explicaciones son la memoria del proyecto: dentro de un año dirán por qué algo está hecho como está, y evitarán que alguien lo «arregle» rompiendo el motivo.

Anexo · Comandos

<code>npm run dev</code>	<code># levantar en desarrollo</code>
--------------------------	---------------------------------------

<code>npx vitest run</code>	<code># las 712 pruebas</code>
-----------------------------	--------------------------------

<code>npx tsc --noEmit</code>	<code># verificar tipos</code>
-------------------------------	--------------------------------

<code>npx eslint src</code>	<code># análisis estático</code>
-----------------------------	----------------------------------

<code>npm run build</code>	<code># compilar para producción</code>
----------------------------	---